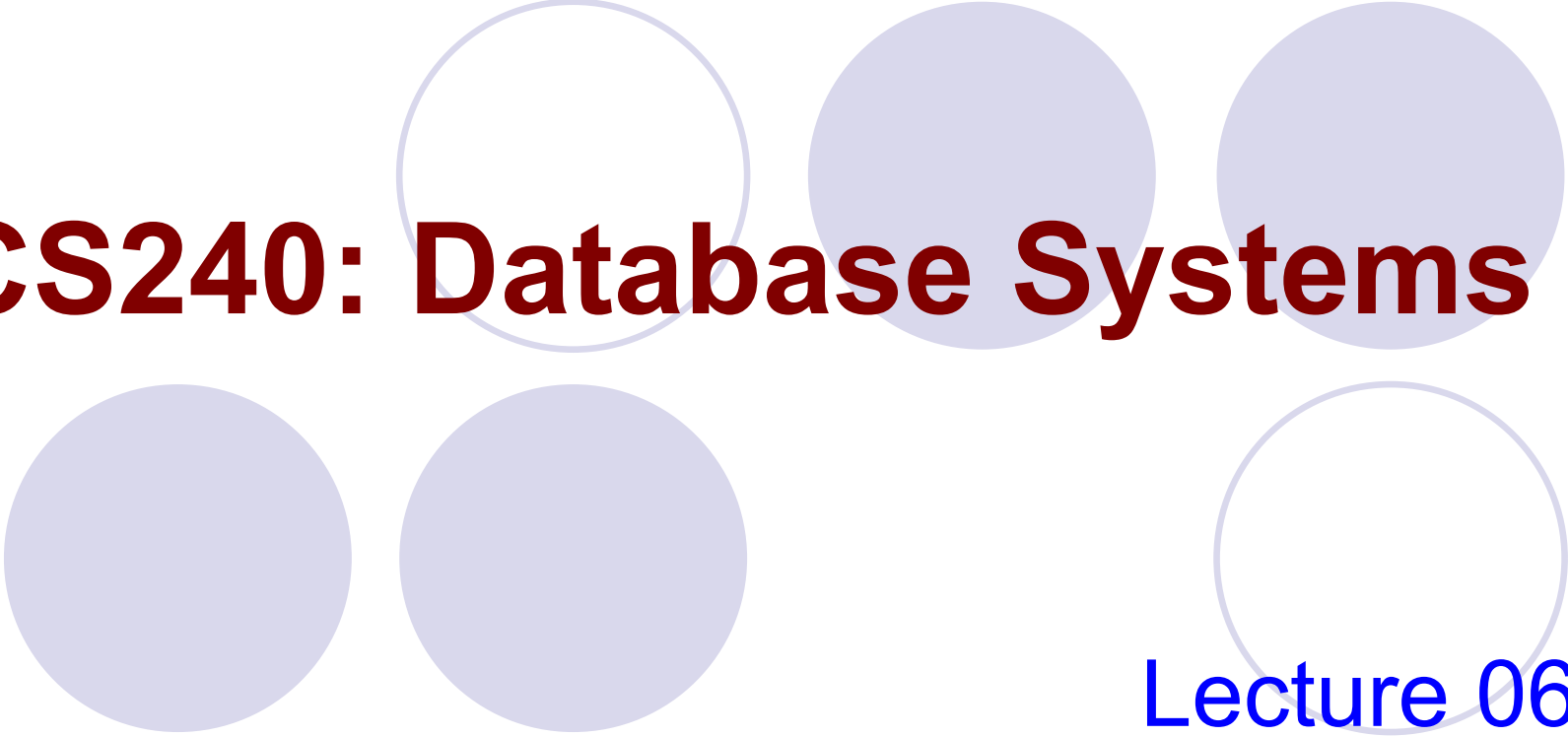




CS240: Database Systems

Lecture 06: Functional Dependencies and Normalization

Outline

- **Problems Caused by Redundancy**
- Functional Dependencies (FDs)
- Normal Forms Based on Keys
- Decomposition
 - Lossless Join Decomposition
 - Dependency Preserving Decomposition
- Normalization
 - BCNF Decomposition Algorithm
 - 3NF Decomposition Algorithm

Problems Caused by Redundancy

- *Waste of Resources of Storage*

Student-ID	Student-Name	Course-ID	Course-Name
123	Tom	COMP231	Database
123	Tom	COMP170	D. Math
567	Mary	COMP104	C++
567	Mary	COMP271	Algorithm
999	Peter	COMP231	Database

Redundancy!
Waste of resources

Problems Caused by Redundancy

- *Redundant Storage* - Some information is stored repeatedly
- *Update Anomalies* - If one copy of such repeated data is updated, an inconsistency is created, unless all copies are similarly updated
- *Insertion Anomalies* - It may not be possible to store certain information unless some other, unrelated, information is stored
- *Deletion Anomalies* - It may not be possible to delete certain information without losing some other, unrelated, information

Problems Caused by Redundancy

Student-ID	Student-Name	Course-ID	Course-Name
123	Tom	COMP231	Database
123	Tom	COMP170	D. Math
567	Mary	COMP104	C++
567	Mary	COMP271	Algorithm
999	Peter	COMP231	Database

Update Anomalies

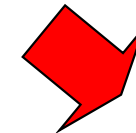
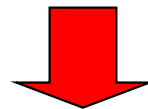
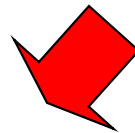
Decompositions

- Intuitively, *redundancy* arise when a relational schema forces an association between attributes that is not natural
- *Functional dependencies* can be used to identify such situations and suggest refinements to the schema
- The essential idea is that many problems arising from redundancy can be addressed by replacing a relation with a collection of “smaller” relations
- A *decomposition* of a relation schema R consists of replacing the relation schema by two (or more) relation schemas so that each contains a subset of attributes of R and together include all attributes in R

Decompositions

- Redundant Storage
- Update Anomalies
- Insertion Anomalies
- Deletion Anomalies

Student-ID	Student-Name	Course-ID	Course-Name
123	Tom	COMP231	Database
123	Tom	COMP170	D. Math
567	Mary	COMP104	C++
567	Mary	COMP271	Algorithm
999	Peter	COMP231	Database



Student-ID	Student-Name
123	Tom
567	Mary
999	Peter

Student-ID	Course-ID
123	COMP231
123	COMP170
567	COMP104
567	COMP271
999	COMP231

Course-ID	Course-Name
COMP231	Database
COMP170	D. Math
COMP104	C++
COMP271	Algorithm

No Potential Inconsistency

Outline

- Problems Caused by Redundancy
- **Functional Dependencies (FDs)**
- Normal Forms Based on Keys
- Decomposition
 - Lossless Join Decomposition
 - Dependency Preserving Decomposition
- Normalization
 - BCNF Decomposition Algorithm
 - 3NF Decomposition Algorithm

Functional Dependencies

- A functional dependency (FD) is a kind of Integrity Constraint that generalizes the concept of a key

Let R be a relation schema and let $X \subset R$ and $Y \subset R$ be nonempty sets of attributes in R . An instance (relation) r of R satisfies the FD $X \rightarrow Y$. If the following holds for every pair of tuples t_1 and t_2 in r

If $t_1.X = t_2.X$, then $t_1.Y = t_2.Y$

The notation $t_1.X$ refers to the projection of tuple t_1 onto the attributes in X

- An FD $X \rightarrow Y$ essentially says that if two tuples agree on the values in attributes X , they must also agree on the values in attributes Y .

Functional Dependencies

- To check $X \rightarrow Y$, erase all other columns; for all rows t_1 ,

t_2

	X_1	...	X_m		Y_1	...	Y_n	
t_1								
t_2								

if t_1, t_2 agree here then t_1, t_2 agree here

- if $X \rightarrow Y$ is a well-defined function thus, functional dependency

Example: Functional Dependencies

Can you find a functional dependency that satisfy the following relation?

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4

Example: Functional Dependencies

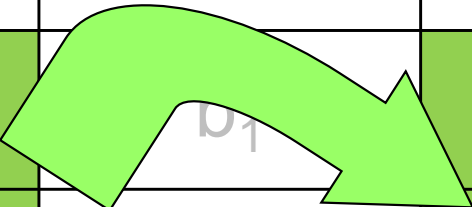
$A \rightarrow C$ is satisfied

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4

Example: Functional Dependencies

$A \rightarrow C$ is satisfied

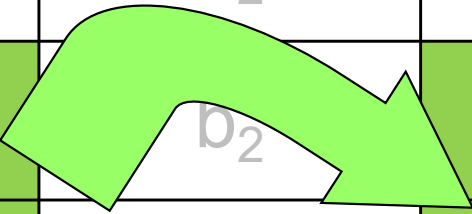
	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4



Example: Functional Dependencies

$A \rightarrow C$ is satisfied

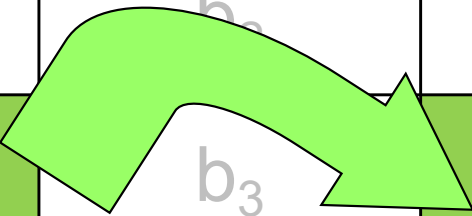
	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4



Example: Functional Dependencies

$A \rightarrow C$ is satisfied

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_3	c_2	d_3
5	a_3	b_3	c_2	d_4



Example: Functional Dependencies

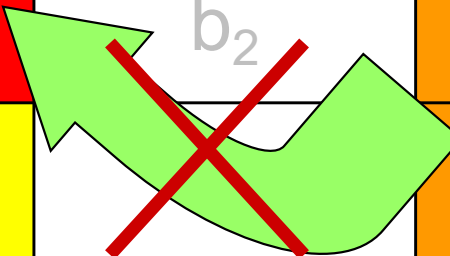
Is $C \rightarrow A$ satisfied?

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4

Example: Functional Dependencies

Is $C \rightarrow A$ satisfied? >>> **NO!** <<<

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3		c_2	d_4



Example: Functional Dependencies

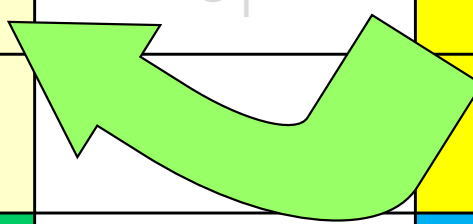
Can you find another functional dependency that satisfies the following relation?

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2	c_2	d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4

Example: Functional Dependencies

$D \rightarrow B$ is also satisfied

	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_2	c_1	d_2
3	a_2	b_2		d_2
4	a_2	b_2	c_2	d_3
5	a_3	b_3	c_2	d_4



Trivial and Non-Trivial FD

- If any two rows *never* agree on the X value, then $X \rightarrow Y$ is *Trivially Preserved*
- *Trivial Functional Dependency*
 - $X \rightarrow Y$ where $Y \subseteq X$
 - e.g, $AB \rightarrow A$
- Non-Trivial Functional Dependency
 - e.g. $AB \rightarrow C$

Example: Trivial and Non-Trivial FD

Student-ID	Student-Name	Course-ID	Course-Name
123	Tom	COMP231	Database
567	Mary	COMP231	Database
999	Peter	COMP271	Algorithm
123	Tom	COMP271	Algorithm

- Student-ID $\rightarrow$ Student-Name (*not trivial dependency*)
- Course-ID $\rightarrow$ Course-Name (*not trivial dependency*)
- Student-Name $\rightarrow$ Student-Name (*trivial dependency*)
- Student-Name, Course-Name $\rightarrow$ Student-Name (*trivial dependency*)
- Student-ID, Course-ID $\rightarrow$
Student-ID, Student-Name, Course-ID, Course-Name
(*not trivial dependency*)

Information Deduced from FDs

- α is a **superkey** for R iff $\alpha \rightarrow R$
 - where R is taken as the schema for relation R
- α is a **candidate key** for R iff
 - $\alpha \rightarrow R$, and
 - for no γ that is a proper subset of α , $\gamma \rightarrow R$

Example

- $AC \rightarrow R, AD \rightarrow R, ACD \rightarrow R$

A	B	C	D
a_1	b_2	c_1	d_2
a_2	b_2	c_1	d_2
a_1	b_4	c_3	d_3
a_4	b_4	c_3	d_4

Example

- $AC \rightarrow R, AD \rightarrow R, ACD \rightarrow R$
- Is AC a candidate key?

A	B	C	D
a_1	b_2	c_1	d_2
a_2	b_2	c_1	d_2
a_1	b_4	c_3	d_3
a_4	b_4	c_3	d_4

Example

A	B	C	D
a_1	b_2	c_1	d_2
a_2	b_2	c_1	d_2
a_1	b_4	c_3	d_3
a_4	b_4	c_3	d_4

- $AC \rightarrow R, AD \rightarrow R, ACD \rightarrow R$
- AC is a candidate key
- AD is *also* a candidate key

Example

A	B	C	D
a ₁	b ₂	c ₁	d ₂
a ₂	b ₂	c ₁	d ₂
a ₁	b ₄	c ₃	d ₃
a ₄	b ₄	c ₃	d ₄

- $AC \rightarrow R, AD \rightarrow R, ACD \rightarrow R$
- AC is a candidate key
- AD is *also* a candidate key
- but, ACD is *not* a candidate key

Note

- A functional dependency must be identified based on semantics of application, not on individual relation instances
 - $A \rightarrow C$ is a functional dependency only if $A \rightarrow C$ is satisfied for **all** relations for a particular schema

Student-ID	Student-Name	Course-ID	Course-Name
123	Tom	COMP231	Database
567	Mary	COMP271	Algorithm
999	Peter	COMP291	C++
567	John	COMP301	Computer Vision

Course-ID $\rightarrow$ Student-Name? – Not make sense

Example: Functional Dependencies

- Application is to keep track of information about employees in a company
- Information to be kept track of includes:
 - eid: employee's id number
 - ename: employee name
 - address: address of the employee
 - sex: employee's sex
 - dname name of the department that the employee works for
 - dhname: department head's name
 - dhsex: department head's sex

Example: Functional Dependencies

- Let's construct a relation schema as follows:

Employee	eid	ename	address	sex	dname	dhname	dhsex
----------	-----	-------	---------	-----	-------	--------	-------

- Which of the following dependencies are true?

1. $\text{eid} \rightarrow \text{ename}$ (T)
2. $\text{ename} \rightarrow \text{eid}$ (F)
3. $\text{eid} \rightarrow \text{dhname}$ (T)
4. $\text{eid} \rightarrow \text{dhsex}$ (T)
5. $\text{sex} \rightarrow \text{address}$ (F)
6. $\text{dname} \rightarrow \text{dhname}$ (T)
7. $\text{dhname} \rightarrow \text{dname}$ (F)

Given

a: Employee's id number is unique

b: Each employee works for only one dept

c: Each department has only one head

Implicit: common sense

a: $\text{eid} \rightarrow \text{eid}, \text{ename}, \text{address}, \text{sex}$

b: $\text{eid} \rightarrow \text{dname}$

c: $\text{dname} \rightarrow \text{dhname}$

Implicit: $\text{dhname} \rightarrow \text{dhsex}$

Reasoning about FDs

- F - a set of functional dependencies
- f - an individual functional dependency

f is implied by F if

- whenever all functional dependencies in F are true
then f is true
- e.g, $F = \{A \rightarrow B, B \rightarrow C\}$ implies $A \rightarrow C$

Closure of F

- $F = \{ f_1, f_2, \dots, f_i, \dots, f_n \}$, where $f_1, f_2, \dots, f_i, \dots, f_n$ are functional dependencies
- f – an individual functional dependency
- F *implies* f , if whenever all $f_1, \dots, f_n$ are true, f is true
 - e.g. $F = \{ A \rightarrow B, B \rightarrow C \}$, F implies $A \rightarrow C$ (by transitivity)
- F^+ – *closure* of F : the set of all functional dependencies that F implies
 - e.g. $F = \{ A \rightarrow B \}$, $F^+ = \{ A \rightarrow A, B \rightarrow B, AB \rightarrow AB, A \rightarrow B, A \rightarrow AB, AB \rightarrow A, AB \rightarrow B \}$

Closure of a set of FDs

- The set of all FDs implied by a given set F of FDs is called the closure of F , denoted as F^+
- **Armstrong's Axioms**, can be applied repeatedly to infer all FDs implied by a set of FDs

Suppose X, Y , and Z are sets of attributes over a relation.

Armstrong's Axioms

- Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$
e.g., $\{A, C\} \subseteq \{A, B, C\} \Rightarrow ABC \rightarrow AC$
- Augmentation: if $X \rightarrow Y$, then $XZ \rightarrow YZ$
e.g., $A \rightarrow B \Rightarrow CA \rightarrow CB$
- Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
e.g., $A \rightarrow BD, BD \rightarrow C \Rightarrow A \rightarrow C$

Armstrong's Axioms

- Armstrong's Axioms is sound and complete.
 - Sound: they generate only FDs in F^+
 - Complete: repeated application of these rules will generate all FDs in F^+
- The proof of soundness is straight forward, but completeness is harder to prove

Additional Rules

- Sometimes, it is convenient to use some additional rules while reasoning about F^+

Suppose X, Y , and Z are sets of attributes over a relation.

- Union: if $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
- Decomposition: if $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

- These additional rules are not essential in the sense that their soundness can be proved using Armstrong's Axioms

Example: Closure of F

$R = (A, B, C)$

$F = \{ A \rightarrow B, B \rightarrow C \}$

$F^+ = \{$
 $A \rightarrow A, B \rightarrow B, C \rightarrow C,$
 $AB \rightarrow AB, BC \rightarrow BC, AC \rightarrow AC, ABC \rightarrow ABC,$
 $AB \rightarrow A, AB \rightarrow B,$
 $BC \rightarrow B, BC \rightarrow C,$
 $AC \rightarrow A, AC \rightarrow C,$
 $ABC \rightarrow AB, ABC \rightarrow BC, ABC \rightarrow AC,$
 $ABC \rightarrow A, ABC \rightarrow B, ABC \rightarrow C,$

Using reflexivity, we
can generate all
trivial dependencies

$A \rightarrow B, \quad \dots (1) \text{ (given)}$

$B \rightarrow C, \quad \dots (2) \text{ (given)}$

$A \rightarrow C, \quad \dots (3) \text{ (transitivity on (1) and (2))}$

$AC \rightarrow BC, \quad \dots (4) \text{ (augmentation on (1))}$

$AC \rightarrow B, \quad \dots (5) \text{ (decomposition on (4))}$

$A \rightarrow AB, \quad \dots (6) \text{ (augmentation on (1))}$

$AB \rightarrow AC, AB \rightarrow C, B \rightarrow BC,$

$A \rightarrow AC, AB \rightarrow BC, AB \rightarrow ABC, AC \rightarrow ABC, A \rightarrow BC, A \rightarrow ABC \}$

Attribute Closure

- Computing the closure of a set of FDs can be expensive
- In many cases, we just want to check if a given FD

$X \rightarrow Y$ is in F^+

X - a set of attributes

F - a set of functional dependencies

- **Attribute Closure X^+** with respect to F , is the set of attributes A such that $X \rightarrow A$ can be inferred using the Armstrong's Axioms.

Given a set of attributes $A_1, \dots, A_n$

The **closure**, $\{A_1, \dots, A_n\}^+ = \{B \in \text{Atts} \mid A_1, \dots, A_n \rightarrow B\}$

Algorithm: Attribute Closure

- α – a set of attributes
- α^+ – closure of α under F
- Algorithm to compute α^+ :

closure $:= \alpha$

repeat until (there is no change) do

 for each $\beta \rightarrow \gamma$ do

 if $\beta \subseteq \text{closure}$ then closure $:= \text{closure} \cup \gamma$

 end for

Example: Attribute Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- To compute AG^+ :

closure = AG

closure = ABG ($A \rightarrow B$)

closure = ABCG ($A \rightarrow C$)

closure = ABCGH ($CG \rightarrow H$)

closure = ABCGHI ($CG \rightarrow I$)

Is AG a candidate key?

$AG \rightarrow R$

$A \rightarrow R?$

$G \rightarrow R?$

- To check if a given FD $AG \rightarrow HI$ is in F^+ , Check if HI is in AG^+

Example: Attribute Closure

- $R = (A, B, C, D)$
- $F = \{ A \rightarrow B, B \rightarrow C \}$

$A^+, B^+, C^+, AB^+?$

Outline

- Problems Caused by Redundancy
- Functional Dependencies (FDs)
- **Normal Forms Based on Keys**
- Decomposition
 - Lossless Join Decomposition
 - Dependency Preserving Decomposition
- Normalization
 - BCNF Decomposition Algorithm
 - 3NF Decomposition Algorithm

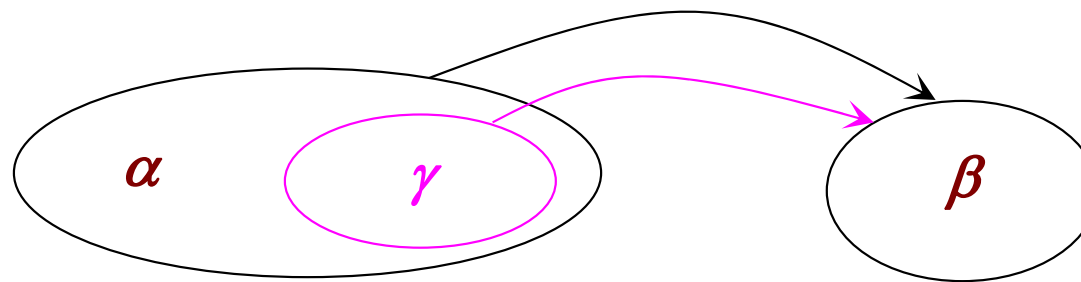
Normal Forms

1 st Normal Form	No repeating data groups
2 nd Normal Form	No partial key dependency
3 rd Normal Form	No transitive dependency
Boyce-Codd Normal Form	Reduce keys dependency
4 th Normal Form	No multi-valued dependency
5 th Normal Form	No join dependency

$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$

Normal Forms

- A functional dependency $\alpha \rightarrow \beta$ is called a **partial dependency** if there is a proper subset γ of α such that $\gamma \rightarrow \beta$; we say that β is partially dependent on α .



Partial dependency

- For example
- **SCE**(studentNo, studentName, courseNo, courseName, score)

studentNo $\rightarrow$ studentName

courseNo $\rightarrow$ courseName

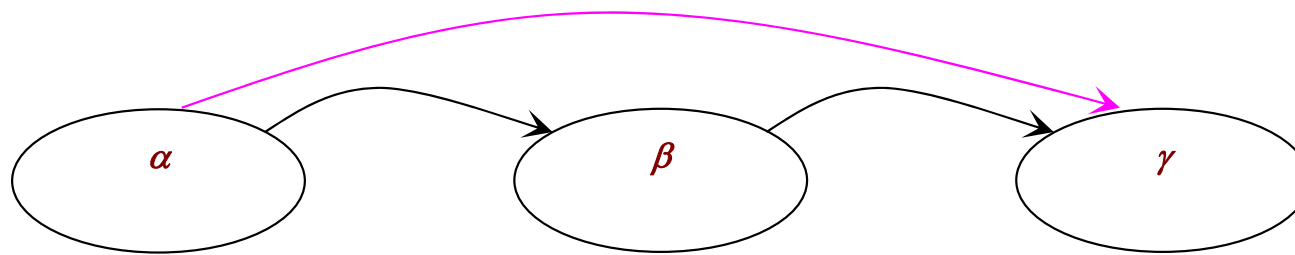
{studentNo, courseNo} $\rightarrow$ score

{studentNo, courseNo} $\rightarrow$ studentName
{studentNo, courseNo} $\rightarrow$ courseName

Partial dependency

Normal Forms

- Let α and β be sets of attributes such that $\alpha \rightarrow \beta$ holds, but $\beta \rightarrow \alpha$ does not hold. Let γ be an attribute that is not in α , is not in β , and for which $\beta \rightarrow \gamma$ holds. A functional dependency $\alpha \rightarrow \gamma$ is called a *transitively dependency*.



Transitively dependency

- For example
- SCI(studentNo, classNo, className, institute)

studentNo $\rightarrow$ classNo
classNo $\rightarrow$ className
classNo $\rightarrow$ institute

studentNo $\rightarrow$ className
studentNo $\rightarrow$ institute

transitively dependency

Normal Forms

- First Normal Form
 - Every field contains only atomic values
 - No lists or sets
 - Implicit in our definition of the relational model
- Second Normal Form
 - Is in 1NF
 - Every non-key attribute (*non-prime attribute*) is fully functionally dependent on the **ENTIRE** primary key (*any candidate key*), i.e., *no partial key dependency*
 - Mainly of historical interest

NOTE: non-prime attribute, an attribute does not belong to any candidate key

Normal Forms

- Third Normal Form
 - Is in 2NF
 - Every non-key attribute (*non-prime attribute*) is *non-transitively dependent* on a primary key (*any candidate key*), i.e., *no transitive dependency*
- Boyce-Codd Normal Form
 - Is in 3NF
 - *Left side* of any non-trivial FD in the table must be a primary key (*a candidate key*)

NOTE: non-prime attribute, an attribute does not belong to any candidate key

Example: 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

IS 1NF

Not 1NF

Example: 2NF

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



$\{Ssn, Pnumber\} \rightarrow Hours$

$Ssn \rightarrow Ename$

$Pnumber \rightarrow Pname, Plocation$

$\{Ssn, Pnumber\} \rightarrow Ename$

$\{Ssn, Pnumber\} \rightarrow Pname$

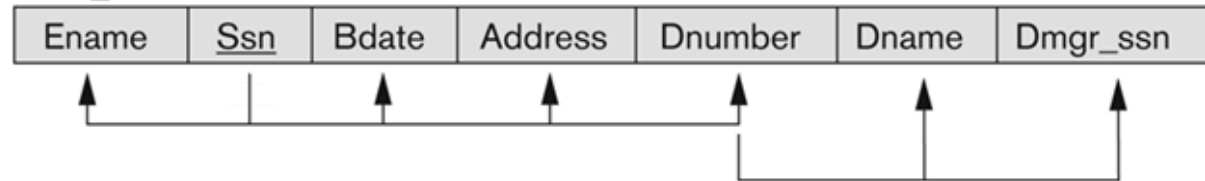
$\{Ssn, Pnumber\} \rightarrow Plocation$

Partial Dependencies
(not in 2NF)

Example: 3NF

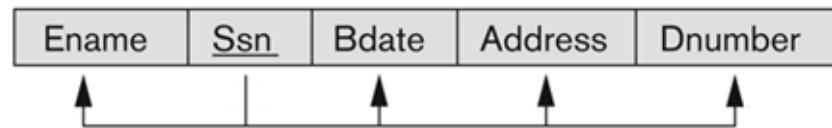
(b)

EMP_DEPT

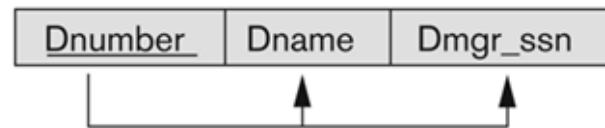


3NF Normalization

ED1



ED2



$Ssn \rightarrow Ename, Bdate, Address, Dnumber$

$Dnumber \rightarrow Dname, Dmgr_ssn$

$Ssn \rightarrow Dname$

$Ssn \rightarrow Dmgr_ssn$

} Transitive Dependencies
(not in 3NF)

Boyce-Codd Normal Form (BCNF)

- R – a relation schema
- F – a set of functional dependencies on R
- R is in BCNF if for any $\alpha \rightarrow A$ in F
 - $\alpha \rightarrow A$ is trivial ($A \in \alpha$), *or*
 - α is a key for R

Example: BCNF

R is in BCNF if for any $\alpha \rightarrow A$ in F

- $\alpha \rightarrow A$ is trivial ($A \in \alpha$), or
- α is a key for R

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Key = { A }
- R is **not** in BCNF, why?
 - $F = \{A \rightarrow B, B \rightarrow C\}$
 - B is not a key,
 - $\{C\} \not\subseteq \{B\}$ (non-trivial)
 - Decompose R into $R_1 = (A, B)$, $R_2 = (B, C)$, then R_1 , R_2 are in BCNF

A	B	C
a1	b1	c1
a2	b1	c1
a3	b1	c1
a4	b2	c2

A	B
a1	b1
a2	b1
a3	b1
a4	b2

$A \rightarrow B$

B	C
b1	c1
b2	c2

$B \rightarrow C$

Example: BCNF

R is in BCNF if for any $\alpha \rightarrow A$ in F

- $\alpha \rightarrow A$ is trivial ($A \in \alpha$), or
- α is a key for R

- $R = (A, B, C)$
- $F = \{A \rightarrow B, C \rightarrow B\}$
- Key = { AC }
- R is **not** in BCNF
- Decomposition into
 - $R_1 = (A, B), R_2 = (B, C)$
 - R_1 and R_2 are in BCNF
 - What is the result when we join R_1 and R_2 ?

A	B	C
a1	b1	c1
a1	b1	c3
a3	b1	c1
a4	b2	c2

A	B
a1	b1
a3	b1
a4	b2

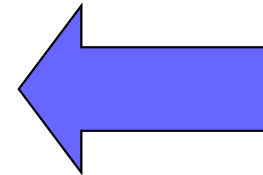
$A \rightarrow B$

B	C
b1	c1
b1	c3
b2	c2

$C \rightarrow B$

Third Normal Form (3NF)

- R – a relation schema
- F – a set of functional dependencies on R
- A – a single attribute in R
- R is in 3NF if for any $\alpha \rightarrow A$ in F
 - $\alpha \rightarrow A$ is trivial ($A \in \alpha$), or
 - α is a key for R, or
 - A is *part of some key(s)* for R
 - *part of some key(s)*: AB is a key for R
 - then A is a part of the key
 - and B is also a part of the key
- R is in BCNF $\Rightarrow$ R is in 3NF



Example

R is in 3NF if for any $\alpha \rightarrow A$ in F

- $\alpha \rightarrow A$ is trivial ($A \in \alpha$), or
- α is a key for R, or
- A is *part of* some key(s) for R

- $R = (A, B, C, D, E)$
- $F = \{ AE \rightarrow BCD, D \rightarrow A \}$
- Key = { AE, DE }
- R is in 3NF
- Is R in BCNF?
 - No. D is not a key for R
 - BCNF implies 3NF but 3NF cannot imply BCNF

Motivation of 3NF

- By making an exception for certain dependencies involving key attributes, we can ensure that every relation schema can be decomposed into a collection of 3NF relations using only *lossless-join*, *dependency-preserving* decompositions
- Such a guarantee does not exist for BCNF relations.
- Unlike BCNF, *some redundancy* is possible with 3NF